

★ ARRAYS & HASHING - CHEAT SHEET (VOLUME 1)

1. INTERVIEW FLOW

- 1 Clarify**
Constraints, edge cases, input range, duplicates?
- ↓
- 2 Brute Force**
Write simple solution
- ↓
- 3 Better**
Optimize Time
- ↓
- 4 Optimal**
Best approach
- ↓
- 5 Complexity**
Time & Space
- ↓
- 6 Code**
Clean & bug free
- ↓
- 7 Explain**
Walk through

2. PATTERN RECOGNITION

🛡️	Need uniqueness?	HashSet
📊	Need frequency count?	HashMap
abc	Small fixed alphabet (a-z)?	int[26]
👥	Need grouping?	HashMap<Key, List>
🏆	Need Top K frequent?	HashMap + Heap
</>	Need Left & Right products?	Prefix / Suffix
🔵	Need sequence (consecutive)?	HashSet
#	Need matrix validation?	HashSet

3. PROBLEMS COVERED

LC	Problem	Pattern	One-line Memory Trick
217	Contains Duplicate	HashSet	Duplicate ⇒ !set.add()
242	Valid Anagram	Freq Array	Increment one, decrement other
1	Two Sum	HashMap	Target - current
49	Group Anagrams	HashMap + Freq	26-count key
347	Top K Frequent	HashMap + Heap	Frequency → Min Heap
271	Encode & Decode Strings	String Encoding	length#string
238	Product Except Self	Prefix / Suffix	Left × Right
128	Longest Consecutive Seq.	HashSet	Start only if num-1 absent
36	Valid Sudoku	HashSet	Rows, Columns, Boxes

4. BIGGEST AHA MOMENTS

Group Anagrams

key.append(freq).append("#")
avoids ambiguity

Top K Frequent

HashMap → PriorityQueue (min heap)
Keep only K elements

Product Except Self

Brute Force → Left[] → Right[] → Left × Right
Result[] = rightProduct

Longest Consecutive

Sorting? → No → !contains(num-1)

5. COMPLEXITY QUICK REFERENCE

HashSet contains / add / remove	O(1) avg
HashMap get / put / containsKey	O(1) avg
Array access	O(1)
For loop (n times)	O(n)
Two separate loops	O(2n) = O(n)

6. BIG-O RULES

Sequential loops

ADD

```
for i in n
...
for j in n
...
```

$$= O(2n) = O(n)$$

Nested loops

MULTIPLY

```
for i in n
  for j in n
    ...
```

$$= O(n^2)$$

Always drop constants.

7. JAVA APIS USED

HashMap

```
put(key, val)
get(key)
containsKey(key)
getOrDefault(key, def)
computeIfAbsent(key, k -> val)
keySet() / values()
```

HashSet

```
add(val)
contains(val)
remove(val)
```

StringBuilder

```
append(x)
toString()
```

Arrays

```
sort(arr)
```

The if is for avoiding repeated traversals, not for correctness.

Encode & Decode

5#Hello

length#string Never use split().
Parse length first.

Nested loops (n x n)	$O(n^2)$
Sorting	$O(n \log n)$
PriorityQueue offer / poll	$O(\log n)$

Focus on growth.

↪ toString(arr)

8. COMMON CONVERSIONS

String → char[]	s.toCharArray()
char[] → String	new String(char[])
int → String	String.valueOf(Integer.parseInt(i))
String → int	Integer.parseInt(s)
char → int	c - '0'
int → char	(char) (i - '0')

9. LAMBDA EXPRESSIONS (QUICK)

Compute If Absent

```
map.computeIfAbsent(key, k -> new ArrayList<>())
```

Comparator (Min Heap)

```
(a, b) -> a - b
```

Comparator (by frequency)

```
(a, b) -> freqMap.get(a) - freqMap.get(b)
```

Sort (custom)

```
arr.sort((a, b) -> b - a)
```

10. WHEN TO USE WHAT?

Use This	When?
HashSet	Check existence / uniqueness
HashMap	Count frequency / key-value
int[26]	Only lowercase letters a-z
Heap (PQ)	Top K / Priority based
Prefix/Suffix	Left/Right contribution problems
HashSet (Seq)	Consecutive sequence problems
HashSet (Matrix)	Row / Col / Box validation

11. TIPS & TRICKS

- ✓ Think pattern first, code later.
- ✓ **Can HashSet replace sorting? Always ask!**
- ✓ Avoid repeated work (like in Longest Consecutive).
- ✓ Use freq array for small fixed ranges (0-25).
- ✓ For Top K → use Min Heap of size K.
- ✓ Edge cases decide AC/TLE. Handle them early.

12. COMMON PITFALLS

- ✗ Forgetting to handle empty input
- ✗ Not clearing data structure in multiple test cases
- ✗ Using split() in Encode/Decode (TLE)
- ✗ Wrong comparator in PriorityQueue
- ✗ Not checking num-1 before starting sequence (LC 128)

13. MEMORY TRICKS

Set for YES/NO (Exists?)
Map for HOW MANY / **VALUE (Count / Store)**
Heap for **TOP K (Sorted partially)**
Left x Right (Product Except Self)
Start from BEGINNING (num-1 not exists)
Length#String (Encoding rule)

14. COMPLEXITY SUMMARY TABLE

Approach	Time Complexity	Space Complexity	Example Problems
Brute Force (Nested Loop)	$O(n^2)$	$O(1)$	Two Sum, Product Except Self
Hashing (Set/Map)	$O(n)$	$O(n)$	Contains Duplicate, Two Sum
Sorting	$O(n \log n)$	$O(1) / O(n)$	Group Anagrams (alt), Longest Consecutive (alt)

Row, Col, Box (Sudoku rule)

Heap (Top K)	$O(n \log k)$	$O(k)$	Top K Frequent Elements
Prefix / Suffix	$O(n)$	$O(n)$	Product Except Self

🏆 GOLDEN RULE: CLARIFY → THINK → CHOOSE RIGHT DS → OPTIMIZE → CODE → EXPLAIN ⭐

Keep Practicing, Keep Improving, Crack FAANG! 🙌

ARRAYS & HASHING

CHEAT SHEET

CORE IDEA

HashSet → Uniqueness, fast lookups

 **HashMap** → Frequency, mapping, counts

 **Frequency Array** → When range is small

✂ **Sorting vs Hashing** → Hashing is $O(n)$, Sorting is $O(n \log n)$

COMPLEXITY

n = size of input

Operation	HashSet / HashMap	Array / Loop	Sort
Lookup	$O(1)$ avg	$O(n)$	$O(\log n)$
Insert	$O(1)$ avg	$O(1)$	-
Delete	$O(1)$ avg	$O(n)$	-
Traversal	$O(n)$	$O(n)$	$O(n)$
Space	$O(n)$	$O(1)$	$O(1)$

COMMON PATTERNS

✓ Unordered data → **HashSet / HashMap**

 Counting / Frequency → **HashMap**

 Group by something → **HashMap<key, List>**

 Check existence → **HashSet**

✗ Avoid duplicates → **HashSet**

💡 Optimize lookup → Replace sorting with Hashing

KEY PROBLEMS IN ARRAYS & HASHING

#	Problem	Pattern	Key Idea	Time	Space	Key Tip / Code Snippet
✓ 217	Contains Duplicate	HashSet	If size changes → duplicate exists	$O(n)$	$O(n)$	<code>if (!set.add(x)) return true;</code>
✓ 242	Valid Anagram	HashMap / Array[26]	Count freq of both strings	$O(n)$	$O(1/26)$	<code>count[c]++; count[c]-;</code>
✓ 1	Two Sum	HashMap	Store num → index	$O(n)$	$O(n)$	<code>if (map.containsKey(target-num))</code>
✓ 49	Group Anagrams	HashMap	Sort string or count freq as key	$O(n * k \log k)$	$O(n * k)$	<code>map.computeIfAbsent(key, k→ new ArrayList<>())</code>
✓ 347	Top K Frequent	HashMap + Heap	Count freq + Min Heap (size k)	$O(n \log k)$	$O(n)$	<code>pq.offer(x); if (pq.size() > k) pq.poll();</code>
✓ 271	Encode & Decode	String Builder	Length#string format	$O(n)$	$O(n)$	<code>sb.append(s.length()).append("#").append(s);</code>
✓ 238	Product Except Self	Prefix / Suffix	Left & Right products	$O(n)$	$O(1)$	<code>left[i] * right[i]</code>

#	Problem	Pattern	Key Idea	Time	Space	Key Tip / Code Snippet
✓ 128	Longest Consecutive	HashSet	Start from sequence start only	O(n)	O(n)	<code>if (!set.contains(x-1)) { while(set.contains(x+1)) x++; }</code>
✓ 36	Valid Sudoku	HashSet	Row, Col, 3×3 box → no duplicates	O(1) (constant)	O(1)	Use <code>set.clear()</code> for each row/col/box

QUICK TIPS & TRICKS

✓ HashSet Uniqueness

```
Set<Integer> set = new
HashSet<>();
for(int x : nums){
    if(!set.add(x)) return
true; // duplicate
}
```

📊 Frequency with HashMap

```
Map<Integer, Integer> map =
new HashMap<>();
for(int x : nums){
    map.put(x,
map.getDefault(x, 0) + 1);
}
```

🔍 Two Sum Pattern

```
Map<Integer, Integer> map =
new HashMap<>();
for(int i=0; i<nums.length;
i++){
    int need = target -
nums[i];
    if(map.containsKey(need))
return new int[]
{map.get(need), i};
    map.put(nums[i], i);
}
```

🌸 Group Anagrams (Key by Count)

```
int[] count = new int[26];
for(char c : s.toCharArray())
count[c-'a']++;
StringBuilder key = new
StringBuilder();
for(int c : count)
key.append(c).append('#');
map.computeIfAbsent(key.toStri
ng(), k→ new ArrayList<
>()).add(s);
```

🔥 Top K Frequent (Min Heap)

```
PriorityQueue<int[]> pq = new
PriorityQueue<>((a,b)→
a[1] - b[1]); // [num, freq]
for(int num : map.keySet()){
    pq.offer(new int[]{num,
map.get(num)});
    if(pq.size() > k)
pq.poll();
}
```

🏠 Product Except Self (Without Division)

```
int[] res = new int[n];
int left = 1;
for(int i=0; i<n; i++){ res[i]
= left; left *= nums[i]; }
int right = 1;
for(int i=n-1; i≥0; i--){
res[i] *= right; right *=
nums[i]; }
```

🔪 Longest Consecutive Sequence

```
Set<Integer> set = new
HashSet<>(nums);
int longest = 0;
for(int x : set){
    if(!set.contains(x-1)){
        int len = 1;

        while(set.contains(x+1)){
            len++; x++; }
        longest =
Math.max(longest, len);
    }
}
```

📋 Valid Sudoku (3 Steps)

1. Check each row
2. Check each column
3. Check each 3×3 box

```
Use Set<Character> set = new
HashSet<>();
set.clear() after each check.
```

GENERAL REMINDERS

★ Think: Can I use HashSet / HashMap?

- ★ **Hashing beats sorting** when order doesn't matter.
- ★ **Always analyze:** Time & Space.
- ★ **Edge cases** make or break solutions.
- ★ **Clarity > Cleverness** in interviews.

Master the Patterns. Solve with Confidence. Ace the Interviews. 🚀

VOLUME 1: ARRAYS & HASHING

CHEAT SHEET

★ FAANG
Interview Edition🔗 Quick
Lookup⚡ Patterns First,
Code Next1. INTERVIEW
FLOW (ALWAYS
FOLLOW)

1. Clarify

Ask constraints, edge cases, duplicates, range, complexity.



2. Brute Force

Get it working first.



3. Better

Can we optimize?



4. Optimal

Best approach (Time/Space).



5. Complexity

Time and Space analysis.



6. Code

Clean, bug-free code.



7. Explain

Walkthrough + dry run.

2. PATTERN RECOGNITION

Need	Use	Example Problems
Uniqueness / Duplicate	HashSet	217
Frequency / Count	HashMap / int[26]	242, 347
Pair / Complement	HashMap	1
Grouping	HashMap<Key, List>	49
Top K Frequent	HashMap + Min Heap	347
Left & Right Info	Prefix / Suffix	238
Consecutive Sequence	HashSet	128
Matrix Validation	HashSet	36
Encoding / Decoding	Length + Delimiter	271

3. PROBLEMS AT A GLANCE

LC #	Problem	Pattern	Key Idea
217	Contains Duplicate	HashSet	If !set.add(x) ⇒ duplicate
242	Valid Anagram	Frequency Array	Count++ then Count--
1	Two Sum	HashMap	target - num
49	Group Anagrams	HashMap + Freq Sig	26-count key string
347	Top K Frequent	HashMap + Heap	Min Heap of size K
271	Encode & Decode	Length + Delimiter	length#string format
238	Product Except Self	Prefix / Suffix	Left[i] x Right[i]
128	Longest Consecutive	HashSet	Start if num-1 not present
36	Valid Sudoku	HashSet	Rows, Cols, 3×3 boxes

4. KEY PATTERNS & TRICKS (HIGH IMPACT)

4.1 Group Anagrams (LC 49)

```
int[] freq = new
int[26];
for(char c :
s.toCharArray())
    freq[c - 'a']++;
StringBuilder key =
new StringBuilder();
for(int f : freq)
    key.append(f).append('#');
```

💡 # is important to avoid ambiguity.

4.2 Top K Frequent (LC 347)

```
Map<Integer,Integer>
map = new HashMap<>();
// count frequency
PriorityQueue<int[]>
pq = new
PriorityQueue<>()
    (a,b) → a[1] - b[1]
// min heap
);
for(int key :
map.keySet()){
    pq.offer(new int[]
```

4.3 Product Except Self (LC 238)

- Left Product Array
- Right Product (in one pass)
- Result[i] = Left[i] * Right[i]

```
nums: [1,2,3,4]
left: [1,1,2,6]
right: [24,12,4,1]
res: [24,12,8,6]
```

💡 No division. O(n) time, O(1) extra space (excluding output).

4.4 Longest Consecutive (LC 128)

```
Set<Integer> set = new
HashSet<>(nums); //
add all numbers
int longest = 0;
for(int num : set){
    if(!set.contains(num
- 1)){
        int curr = num,
count = 1;
while(set.contains(cur
r + 1)){
```

4.5 Encode & Decode (LC 271)

ENCODE:

```
sb.append(s.length()).appe
nd('#').append(s);
```

DECODE:

Read length until '#', then take that many characters.

Example:

```
Input :
["Hello","World"]
Encode: 5#Hello5#World
```

```
{key, map.get(key)}});
    if(pq.size() > k)
        pq.poll();
}
```

💡 Keep only K elements in heap.

```
curr++; count++;
}
longest =
Math.max(longest,
count);
}
```

💡 Start count only if num-1 is absent.

Decode:
["Hello", "World"]

💡 Never use split(). Parse length.

5. COMPLEXITY QUICK REFERENCE

HashSet add/contains/remove	O(1) avg
HashMap put/get/contains	O(1) avg
Loop (n times)	O(n)
Two separate loops	O(2n) = O(n)
Nested loops (n x m)	O(n*m)
Sorting	O(n log n)
PriorityQueue offer / poll	O(log n)

6. JAVA COLLECTIONS & APIS (MOST USED)

HashSet<Integer> set	HashMap<K,V> map	PriorityQueue<int[]> pq	String StringB
add(x)	put(k,v)	offer(x)	toChar
contains(x)	get(k)	poll()	String.v
remove(x)	containsKey(k)	peek()	Integer
size()	getOrDefault(k, def)	size()	StringB
clear()	computeIfAbsent(k, v→newV)	For min heap: (a,b) -> a[1] - b[1]	sb.app
	keySet(), values(), entrySet()		sb.toSt

7. CONVERSIONS (QUICK RECALL)

String → char[]	s.toCharArray()
char[] → String	new String(arr)
int → String	String.valueOf(x) Integer.toString(x)
String → int	Integer.parseInt(s)
char → int	c - '0'
int → char	(char) (x + '0')
List → Array	list.toArray(new T[0])

8. BIG-O RULES (DON'T FORGET)

Sequential loops
ADD.

Example 1
(Sequential):

```
for i in n
for j in n
// O(n) +
O(n) = O(n)
```

Nested loops
MULTIPLY.

Example 2
(Nested):

```
for i in n
for j in n
n
// O(n * n)
= O(n^2)
```

One loop = O(n), Two loops sequential = O(n)

9. COMMON MISTAKES (AVOID)

- ✗ Using sort when HashSet can solve in O(n).
- ✗ Forgetting to check num-1 before counting sequence.
- ✗ Using split() in Encode/Decode (causes errors with #).
- ✗ Not handling negative numbers or edge cases.
- ✗ Not analyzing complexity.
- ✗ Over-complicating when a simpler pattern fits.

10. INTERVIEW COMMUNICATION TIPS

- ✓ Think out loud. Explain each step.
- ✓ Start with brute force, then optimize.
- ✓ Give examples and dry run.
- ✓ State assumptions clearly.
- ✓ Write clean, readable code.
- ✓ Summarize time & space at the end.

Golden Line:
"Brute Force → Better → Optimal → Complexity → Code → Explain"

MEMORY MANTRAS



Need Unique? Use HashSet Need Count? Use HashMap Need Left & Right? Use Prefix / Suffix Need Top K? Use Heap Need Consecutive? Use HashSet



PRACTICE. PATTERNS.

REPEAT. CRACK FAANG!

ARRAYS & HASHING – PATTERNS CHEAT SHEET

Core Patterns, When to Use, Key
Ideas & Common Formulas

1

DUPLICATE DETECTION HashSet

When to Use

- Find if any element appears more than once.

Key Idea

- Insert into set, if already present → duplicate.

How it Works / Formula / Code

```
Set<Integer> set =
new HashSet<>();
for (int x : nums) {
    if
    (set.contains(x))
    return true;
    set.add(x);
}
return false;
```

Time: $O(n)$ | Space: $O(n)$

Example

Input: [1, 2, 3, 1]
Output: true

1	2	3	1
---	---	---	---

↓
duplicate

2

FREQUENCY COUNTING HashMap / int[26]

When to Use

- Count occurrences of elements.

Key Idea

- Map each element → its frequency.

Num	Freq
1	1
2	2
3	3

How it Works / Formula / Code

A) HashMap (any values)

```
Map<Integer, Integer>
map = new HashMap<>();
for (int x : nums) {
    map.put(x,
    map.getDefault(x,
    0) + 1);
}
```

B) Frequency Array (only for small range e.g. a-z)

```
int[] freq = new
int[26];
for (char c :
s.toCharArray()) {
    freq[c - 'a']++;
}
```

Time: $O(n)$ | Space: $O(n)$

3

COMPLEMENT / PAIR SEARCH HashMap

When to Use

- Find pair with given sum / target.

Key Idea

- For each x, check if (target - x) exists in map.

Example

Input: nums = [2, 7, 11, 15], target = 9
Output: [0, 1]

Idx	0	1
	2	7
	↓ ↑	
	9 - 2 = 7	
	found at idx 1	

How it Works / Formula / Code

```
Map<Integer, Integer>
map = new HashMap<>();
for (int i = 0; i <
nums.length; i++) {
    int complement =
    target - nums[i];
    if
    (map.containsKey(complement))
    return new
    int[]
    {map.get(complement),
    i};
    map.put(nums[i],
    i);
}
return new int[]{};
```

Time: $O(n)$ | Space: $O(n)$

4

GROUPING / BUCKETING HashMap<List>

When to Use

- Group elements by key (e.g., anagrams, indices, values).

Key Idea

- Map key → list of elements.

How it Works / Formula / Code

```
Map<String,
List<String>> map = new
HashMap<>();
for (String s : strs) {
    String key = ...;
    // e.g. sorted string
    map.computeIfAbsent(key,
    k → new ArrayList<>())
    .add(s);
}
```

[eat, ate, tea]	[tan, nat]	[bat]
↓	↓	↓
key: aet	key: ant	key: abt

Time: $O(n * k \log k)$ | Space: $O(n * k)$
(n = #strings, k = avg length)

5

ENCODING / StringBuilder

DECODING STRINGS

When to Use

- Store and retrieve list of strings in single string.

Key Idea

- Prefix each string with its length.

How it Works / Formula / Code

Encode:

```
for (String s : strs)
    sb.append(s.length())
      .append("#").append(s);
```

Decode:

Read number until '#', then next that many chars is the string.

Example

Input: ["we", "love", "leetcode"]
 Encode: "2#we4#love8#leetcode"
 Decode: ["we", "love", "leetcode"]

2 # w e 4

6

PREFIX / SUFFIX PATTERNS

Arrays

When to Use

- Product except self, prefix sums, suffix sums, etc.

Key Idea

- Precompute left and right contributions.

Example

nums = [1,2,3,4]
 ans = [24,12,8,6]

i	nums	left
0	1	1
1	2	1
2	3	2
3	4	6

How it Works / Formula / Code (Product Except Self)

```
left[i] = product of all
nums [0 ... i-1]
right[i] = product of all
nums [i+1 ... n-1]
ans[i] = left[i] * right[i]

int n = nums.length;
int[] left = new int[n],
right = new int[n], ans =
new int[n];
left[0] = 1; right[n-1] =
1;
for (int i = 1; i < n; i++) left[i] = left[i-1] *
nums[i-1];
for (int i = n-2; i ≥ 0; i--) right[i] = right[i+1]
* nums[i+1];
ans[i] = left[i] * right[i];
```

7

LONGEST CONSECUTIVE SEQUENCE

Hashing

When to Use

- Find length of longest consecutive elements.

Key Idea

- Only start counting from the first element of a sequence (i.e., num-1 not present).

Example

Input: [100,4,200,1,3]
 Output: 4
 (1,2,3,4)

100 4
 200 1
 3 2
 length = 4

How it Works / Formula / Code

```
Set<Integer> set = new
HashSet<>();
for (int n : nums)
    set.add(n);
int longest = 0;
for (int x : set) {
    if (!set.contains(x
- 1)) { // start
        int count = 1,
cur = x;
        while
(set.contains(cur + 1))
{
            cur++;
count++;
}
        longest =
count;
```

8

MATRIX VALIDATION (SUDOKU)

HashSet
 (Rows,
 Cols,
 Boxes)

When to Use

- Validate Sudoku board.

Key Idea

- No duplicates in rows, columns and 3×3 boxes.

How it Works

- Check each row using set
- Check each column using set
- Check each 3×3 box using set

3×3 Box Traversal

```
for (int r = 0; r < 9; r
+= 3)
    for (int c = 0; c < 9;
c += 3)
        for (int i = r; i <
r + 3; i++)
            for (int j = c; j
< c + 3; j++)
                validate(board[i][j]);
```

Visual - 3×3 Boxes

(0,0)	(0,3)
(3,0)	(3,3)
(6,0)	(6,3)

WHEN TO USE WHAT?

✓ HashSet	Fast lookup, duplicates, membership
✓ HashMap	Counts, mapping, complement search, grouping, index mapping
✓ Frequency Array	Small fixed range (e.g., a-z, 0-26)
☒ Sorting	Order matters, two pointers after sort, grouping

COMMON HASHING FORMULAS / TRICKS

- Get or Insert:**
`map.getOrDefault(key, 0) + 1`
- Insert List:**
`map.computeIfAbsent(key, k → new ArrayList<>()).add(val)`
- Check Key:**
`map.containsKey(key)`
- Check Value in Set:**
`set.contains(val)`
- Remove Key:**
`map.remove(key)`

CHAR / ASCII TRICKS

- char to index:** `c - 'a' // 0 to 25`
- index to char:** `(char) ('a' + i)`
- char to int digit:** `c - '0'`
- int to char digit:** `(char) ('0' + x)`

TIME COMPLEXITY QUICK GUIDE (Average Case)

Operation	HashSet	HashMap
Add / Put	O(1)	O(1)
Remove	O(1)	O(1)
Contains / Get	O(1)	O(1)
Iterate All	O(n)	O(n)

IMPORTANT NOTES

- Hashing gives O(1) average lookup but O(n) worst case (rare with good hash function).
- Choose the right pattern – it saves time and code.
- Think: Can I transform this into a lookup problem?

↻ **Heap / PQ**

Top K frequent, K largest/smallest

✓ **Arrays**

Prefix/Suffix, sliding window, simulation

- **Iterate Entries:** `for (Map.Entry<K,V> e : map.entrySet())`

- **Convert Values to List:** `new ArrayList<>(map.values())`

- **Sort Map by Value:**

```
map.entrySet().stream()
    .sorted((a,b) → b.getValue() - a.getValue())
    .collect(Collectors.toList());
```

- **Uppercase:**

```
c - 'A'
// 0 to 25
```

**FREQUENCY
ARRAY
TEMPLATE
(a-z)**

```
int[] freq
= new
int[26];
for (char
c :
s.toCharArray())
    freq[c
- 'a']++;
```

★ **Golden Rule:** Understand the Pattern → Choose the Right Data Structure → Code Clean → Analyze Complexity → Optimize if Needed

ARRAYS & HASHING CHEAT SHEET

PROBLEM	KEY IDEA	APPROACH / FORMULA / PATTERN	TIME COMPLEXITY	SPACE COMPLEXITY	KEY TAKEAWAYS / PATTERNS
1. 217 Contains Duplicate (Easy)	Use HashSet to track seen elements.	<pre>Set<Integer> set = new HashSet<>(); for (int x : nums) { if (set.contains(x)) return true; set.add(x); } return false;</pre>	$O(n)$	$O(n)$	- HashSet for fast lookup - Detect duplicates
2. 242 Valid Anagram (Easy)	Two strings are anagrams if both have same character frequency.	<pre>int[] count = new int[26]; for (char c : s.toCharArray()) count[c-'a']++; for (char c : t.toCharArray()) count[c-'a']--; return all count[i] == 0;</pre>	$O(n)$ (n = length of string)	$O(1)$ (26 letters)	- Frequency counting - Array of size 26 - Increment / decrement
3. 1 Two Sum (Easy)	Store number $\rightarrow$ index in a HashMap. For each num, check if target - num exists.	<pre>Map<Integer, Integer> map = new HashMap<>(); for (int i = 0; i < nums.length; i++) { int comp = target - nums[i]; if (map.containsKey(c omp)) return new int[] {map.get(comp), i}; map.put(nums[i], i); }</pre>	$O(n)$ (n = length of array)	$O(n)$	- HashMap for complement - Store index, not value
4. 49 Group Anagrams (Medium)	Sort each string to create a key. Anagrams have same sorted key.	<pre>Map<String, List<String>> map = new HashMap<>(); for (String s : strs) { char[] arr = s.toCharArray(); Arrays.sort(arr); String key = new String(arr); map.computeIfAbsen</pre>	$O(n * k \log k)$ (n = #strings, k = avg length)	$O(n * k)$	- Sorting as key - HashMap<String, List<String>>

PROBLEM	KEY IDEA	APPROACH / FORMULA / PATTERN	TIME COMPLEXITY	SPACE COMPLEXITY	KEY TAKEAWAYS / PATTERNS
		<pre>t(key, k → new ArrayList<> ()).add(s); } return new ArrayList<> (map.values());</pre>			
5. 347 Top K Frequent Elements (Medium)	Count frequency using HashMap, then use bucket sort (freq → list of numbers).	<pre>1. Count freq: Map<Integer, Integer> freq = new HashMap<>(); 2. Bucket sort: List<Integer>[] bucket = new List[nums.length + 1]; for (int num : freq.keySet()) { int f = freq.get(num); if (bucket[f] = null) bucket[f] = new ArrayList<>(); bucket[f].add(num) ; } 3. Traverse from high freq to low, collect k elements.</pre>	$O(n)$ (n = array length)	$O(n)$	- Frequency Map + Bucket Sort - Buckets sized by frequency - Read from high to low
6. 271 Encode and Decode Strings (Medium)	Use length-prefix encoding: length#string to uniquely separate strings.	<pre>Encode: for (String s : strs) sb.append(s.length ()).append("#").ap pend(s); Decode: read number until '#', then next that many chars is the string.</pre>	Encode: $O(n)$ Decode: $O(n)$ (n = total length of all strings)	$O(n)$	- Length-prefix avoids ambiguity - Unique & decodable
7. 238 Product of Array Except Self (Medium)	Use prefix & suffix products. Avoid division.	<pre>int n = nums.length; int[] left = new int[n], right = new int[n], ans = new int[n]; left[0] = 1; for (int i=1; i<n; i++) left[i] = left[i-1] * nums[i-1]; right[n-1] = 1; for (int i=n-2; i ≥ 0; i--) right[i] = right[i+1] *</pre>	$O(n)$	$O(n)$ (for output array)	- Prefix products & suffix products - No division - Left[i] * Right[i]

PROBLEM	KEY IDEA	APPROACH / FORMULA / PATTERN	TIME COMPLEXITY	SPACE COMPLEXITY	KEY TAKEAWAYS / PATTERNS
		<pre> nums[i+1]; for (int i=0; i<n; i++) ans[i] = left[i] * right[i]; </pre>			
8. 128 Longest Consecutive Sequence (Hard)	Use HashSet. Start counting only from the beginning of a sequence (num-1 not present).	<pre> Set<Integer> set = new HashSet<>(); for (int num : nums) set.add(num); int longest = 0; for (int num : set) { if (!set.contains(num - 1)) { // start of sequence int count = 1, cur = num; while (set.contains(cur + 1)) { count++; cur++; } longest = Math.max(longest, count); } } </pre>	O(n)	O(n)	- HashSet for O(1) lookup - Start from sequence beginning (num-1 not present) - Each number visited once
9. 36 Valid Sudoku (Medium)	Check rows, columns and 3×3 sub-boxes using HashSet.	<pre> 1. For each row → use Set<Character> 2. For each column → use Set<Character> 3. For each 3×3 box → use Set<Character> Box loops: for (int r=0; r<9; r+=3) for (int c=0; c<9; c+=3) for (int i=r; i<r+3; i++) for (int j=c; j<c+3; j++) </pre>	O(1) (Board size is fixed 9×9)	O(1) (Fixed size sets)	- Use HashSet for rows, columns and 3×3 boxes - Box traversal: jump by 3

COMMON HASHING DATA STRUCTURES

• HashSet<T>	→ Uniqueness, Fast lookup
• HashMap<K,V>	→ Key-Value, Frequency count

FREQUENCY COUNT PATTERN

```

Map<Integer, Integer> map = new HashMap<>();
for (int num : nums) {
    map.put(num, map.getOrDefault(num, 0) + 1);
}

```

BUCKET SORT (BY FREQUENCY)

```

List<Integer>[] bucket = new List[nums.length + 1];
for (int num : map.keySet()) {
    int f = map.get(num);
    if (bucket[f] == null) bucket[f] = new ArrayList<>();
}

```

• TreeMap<K,V>	→ Sorted keys
• LinkedHashMap	→ Maintains insertion order

3×3 BOX TRAVERSAL (SUDOKU)

```
for (int r = 0; r < 9; r += 3) {  
    for (int c = 0; c < 9; c += 3) {  
        for (int i = r; i < r + 3; i++) {  
            for (int j = c; j < c + 3; j++) {  
                // process board[i][j]  
            }  
        }  
    }  
}
```

```
bucket[f].add(num);  
}
```

COMPLEXITY REMINDER

- **HashSet / HashMap**
Operations (avg):
add, remove, contains,
get, put → $O(1)$
- **Sorting** → $O(n \log n)$
- **Traversal** → $O(n)$

★ **REMEMBER:** Use the right tool for the right problem. Hashing improves lookup from $O(n)$ to $O(1)$ on average.

ARRAYS & HASHING - PATTERNS CHEAT SHEET

Master these patterns. They solve 80% of Array + Hashing problems in interviews.

(FAANG Cheat Sheet)

IMAGE
1

1. DUPLICATE DETECTION

Use Case: Check if any element appears more than once.

Data Structure: **HashSet**

Key Idea: Add elements to a set. If already present → duplicate found.

```
Set<Integer> set =
new HashSet<>();
for (int x : nums)
{
    if
    (!set.add(x))
return true; //
already exists
}
return false;
```

Problems

- 217. Contains Duplicate
- 219. Contains Duplicate II
- 220. Contains Duplicate III

Time Complexity: **O(n)**
Space Complexity: **O(n)**

2. FREQUENCY COUNTING

Use Case: Count frequency of each element / character.

Data Structure: **HashMap / int[26] (for lowercase letters)**

Key Idea: Count occurrences and store. Helps in anagrams, top K freq, majority element etc.

HashMap approach:

```
Map<Integer,
Integer> map = new
HashMap<>();
for (int x : nums)
{
    map.put(x,
map.getOrDefault(x
, 0) + 1);
}
```

Array approach (chars 'a'-'z'):

```
int[] freq = new
int[26];
for (char c :
s.toCharArray())
freq[c - 'a']++;
```

Problems

- 242. Valid Anagram
- 383. Ransom Note
- 49. Group Anagrams
- 347. Top K Frequent
- 169. Majority Element

Time Complexity: **O(n)**
Space Complexity: **O(n)**

3. COMPLEMENT / PAIR SEARCH

Use Case: Find if a pair exists with given sum / target.

Data Structure: **HashMap (value → index)**

Key Idea: For each element x, check if (target - x) exists. Store seen elements with their index.

```
Map<Integer,
Integer> map = new
HashMap<>();
for (int i = 0; i
< nums.length;
i++) {
    int complement
= target -
nums[i];
    if
(map.containsKey(c
omplement))
return new
int[]
{map.get(complemen
t), i};
    map.put(nums[i],
i);
}
return new int[]
{};
```

Problems

- 1. Two Sum
- 167. Two Sum II
- 560. Subarray Sum = K
- 454. 4Sum II

Time Complexity: **O(n)**
Space Complexity: **O(n)**

4. GROUPING / BUCKETING

Use Case: Group elements by a key

Problems

- 49. Group Anagrams

5. ENCODING / SERIALIZATION

Use Case: Encode list of strings / decode back.

Problems

- 271. Encode

6. PREFIX / SUFFIX TECHNIQUE

Use Case: Product except self, prefix sums, suffix sums.

Problems

- 238. Product

(anagram, index diff, etc).

Data Structure:
HashMap<Key, List/Count>

Key Idea:

Use key as group identifier and store list of elements. Very useful in anagram problems.

```
Map<String, List<String>> map = new HashMap<>();
for (String s : strs) {
    char[] arr = s.toCharArray();

    Arrays.sort(arr);
    String key = new String(arr);

    map.computeIfAbsent(key, k -> new ArrayList<>()).add(s);
}
return new ArrayList<>(map.values());
```

- 438. Find All Anagrams in a String
- 128. Longest Consecutive Sequence (variant)

Time

Complexity:
O(n * k log k)
(k = avg length of string)
Space Complexity:
O(n * k)

Pattern:
length#string

Key Idea:

Prefix each string with its length and a delimiter. Avoid ambiguity ("ab#c" vs "a#bc").

```
// Encode
StringBuilder sb = new StringBuilder();
for (String s : strs) sb.append(s.length()).append('#').append(s);

// Decode
List<String> res = new ArrayList<>();
for (int i = 0; i < s.length(); ) {
    int j = i;
    while (s.charAt(j) != '#') j++;
    int len = Integer.parseInt(s.substring(i, j));
    i = j + 1;

    res.add(s.substring(i, i + len));
    i += len;
}
return res;
```

and
Decode
Strings

Time Complexity:
O(n)
Space Complexity:
O(n)

Key Idea:

Precompute left (prefix) and right (suffix) values. Combine to get result in O(n) without extra structures.

```
int n = nums.length;
int[] left = new int[n], right = new int[n], ans = new int[n];
left[0] = 1;
for (int i = 1; i < n; i++) left[i] = left[i-1] * nums[i-1];
right[n-1] = 1;
for (int i = n-2; i >= 0; i--) right[i] = right[i+1] * nums[i+1];
for (int i = 0; i < n; i++) ans[i] = left[i] * right[i];
```

of Array
Except Self
• 724. Find Pivot Index

Time Complexity:
O(n)
Space Complexity:
O(1)
(excluding output array)

7. LONGEST CONSECUTIVE SEQUENCE

Use Case: Find length of longest sequence of consecutive integers.

Data Structure:
HashSet

Key Idea:

Store all numbers in a set. Start counting only from the beginning of a sequence (i.e., num-1 not present) to avoid duplicates.

```
Set<Integer> set = new HashSet<>();
for (int x : nums) set.add(x);
int longest = 0;
for (int x : set) {
    if
```

Problems

- 128. Longest Consecutive Sequence

Time

Complexity:
O(n)
Space Complexity:
O(n)

8. MATRIX VALIDATION (USING HASHING)

Use Case: Validate rows, columns, and sub-boxes (e.g., Sudoku).

Key Idea:

Use HashSet for each row, column and 3x3 box to check duplicates.

Rows: For each row i, use a set. Check all 9 elements.
Columns: For each col j, use a set. Check all 9 elements.
3x3 Boxes: For top-left corners (0,0), (0,3), (0,6), (3,0), (3,3), (3,6), (6,0), (6,3), (6,6) check 3x3 elements.

Time Complexity: O(1) → fixed 9x9 board (General: O(n^2))

Problem

- 36. Va

5	3
2	2
.	1
1	1
1	1
.	3
1	3
.	2
.	1

WHEN TO USE WHAT?

Scenario	Best Choice
Check existence / duplicates	HashSet
Count frequency / occurrence	HashMap or Frequency Array
Find pair / complement	HashMap
Group by key	HashMap<Key, List>
Top K frequent elements	HashMap + Heap (PriorityQueue)
Range / consecutive sequence	HashSet (sequence start)
Product except self / prefix tasks	Prefix & Suffix Arrays
Matrix / Sudoku validation	HashSet per row/col/box

```
(!set.contains(x - 1)) { // start of sequence
    int cur = x, count = 1;
    while (set.contains(cur + 1)) { cur++; count++; }
    longest = Math.max(longest, count);
}
return longest;
```

Space Complexity: $O(1)$ 

- **Always think:** Can I get $O(1)$ average lookup? → Use HashSet / HashMap
- Read constraints carefully (size, value range, duplicates allowed?).
- Choose the simplest DS that fits the problem.
- Pay attention to space vs time trade-off.



map.put(key, val) → insert/update
map.get(key) → get value (null if not present)
map.containsKey(key) → check key exists
map.computeIfAbsent(key, k → new ArrayList<>()).add(val);



Operation	HashMap / HashSet	Array	Sorting
Lookup	$O(1)$ avg	$O(1)$	$O(\log n)$
Insert	$O(1)$ avg	$O(1)$ amortized	N/A
Delete	$O(1)$ avg	$O(n)$	N/A
Iteration	$O(n)$	$O(n)$	$O(n \log n)$



If n is the size of input:

- HashSet / HashMap → $O(n)$
- Frequency Array (26) → $O(1)$
- Prefix / Suffix → $O(n)$ or $O(1)$ (extra space excluding output)



REMEMBER: Patterns > Problems. Master the pattern, solve any problem!

ARRAYS & HASHING – PROBLEMS, COMPLEXITY & TEMPLATES CHEAT SHEET (IMAGE 3)

👉 Problems → Approach
/ Key Idea → Time Complexity

A. PROBLEM WISE QUICK REFERENCE

#	PROBLEM	KEY IDEA / APPROACH	TIME	SPACE	IMPORTANT NOTES / PATTERN
1	217. Contains Duplicate	HashSet - add while iterating, if already exists → duplicate.	O(n)	O(n)	Typical use: seen/visited tracking
2	242. Valid Anagram	Frequency count (int[26]) or HashMap<Character, Integer>	O(n)	O(1)	Count & compare all frequencies
3	1. Two Sum	HashMap (value → index). Check complement (target - num).	O(n)	O(n)	Store index, not just value
4	49. Group Anagrams	Key = sorted string or 26-char frequency string. HashMap<Key, List<String>>	O(n * k log k)	O(n * k)	k = avg length of string
5	347. Top K Frequent Elements	HashMap for freq + Bucket Sort (freq → list) or Min-Heap (size k).	O(n) Heap: O(n log k)	O(n)	Bucket sort is O(n) time

B. TEMPLATES YOU SHOULD REMEMBER

1. HASHSET - DUPLICATE CHECK

```
Set<Integer> set = new HashSet<>();
for (int x : nums) {
    if (!set.add(x)) return true; // already exists
}
return false;
```

2. HASHMAP - FREQUENCY COUNT

```
Map<Integer, Integer> map = new HashMap<>();
for (int x : nums) {
    map.put(x, map.getOrDefault(x, 0) + 1);
}
```

3. TWO SUM (HASHMAP)

```
Map<Integer, Integer> map = new HashMap<>();
for (int i = 0; i < nums.length; i++) {
    int need = target - nums[i];
    if (map.containsKey(need)) return new int[]{map.get(need), i};
    map.put(nums[i], i);
}
return new int[0];
```

4. GROUP ANAGRAMS (SORTED STRING KEY)

```
Map<String, List<String>> map = new HashMap<>();
for (String s : strs) {
    char[] arr = s.toCharArray();
    Arrays.sort(arr);
    String key = new String(arr);
    map.computeIfAbsent(key, k → new ArrayList<>()).add(s);
}
return new ArrayList<>(map.values());
```

5. TOP K FREQUENT (BUCKET SORT)

```
Map<Integer, Integer> freq = new HashMap<>();
for (int x : nums) freq.put(x, freq.getOrDefault(x, 0) + 1);
List<Integer>
```

6. LONGEST CONSECUTIVE SEQUENCE

```
Set<Integer> set = new HashSet<>(nums);
int longest = 0;
for (int num : set) {
    if (!set.contains(num - 1)) {
        // start of sequence
        int cur = num, len = 1;
        while
```

C. COMMON PATTERNS SUMMARY

PATTERN	HOW TO THINK
Seen / Visited	Use HashSet
Counting / Frequency	HashMap or int[26]
Complement Search	HashMap (value → index)
Grouping	HashMap<Key, List>
Top K / Min/Max K	Heap or Bucket Sort
Consecutive Sequence	HashSet + check (num-1)
Matrix Validation	Separate set for row/col/box
Prefix / Suffix	Two passes (left → right, right → left)
Encoding	length#string
Avoid Division	Prefix & suffix products
Alphabet Count	int[26] with (c - 'a')

D. COMPLEXITY QUICK GUIDE

OPERATION	TIME COMPLEXITY
HashSet add/contains/remove	O(1) avg
HashMap get/put/containsKey	O(1) avg
Array access	O(1)
Traverse Array	O(n)

#	PROBLEM	KEY IDEA / APPROACH	TIME	SPACE	IMPORTANT NOTES / PATTERN
6	271. Encode and Decode Strings	Encode: <code>len#string</code> Decode: read length, then string	$O(n)$	$O(n)$	# as delimiter avoids ambiguity
7	238. Product of Array Except Self	Prefix products (left) + Suffix products (right) No division allowed.	$O(n)$	$O(n)$	<code>res[i] = left[i-1] * right[i+1]</code>
8	128. Longest Consecutive Sequence	HashSet - store all numbers. Start count only if <code>!contains(num-1)</code>	$O(n)$	$O(n)$	Avoid duplicates by this trick
9	36. Valid Sudoku	HashSet for Rows, Columns and 3x3 boxes	$O(1)$ (fixed 9x9 board)	$O(1)$	3 loops → rows, cols, boxes
10	Bonus Pattern: Character Count (lowercase)	Use <code>int[26]</code> , index = <code>char - 'a'</code>	$O(n)$	$O(1)$	Fastest for lowercase letters

```

bucket = new List[nums.length + 1];
for (int num : freq.keySet()) {
    int f = freq.get(num);
    if (bucket[f] == null) bucket[f] = new ArrayList<>();
    bucket[f].add(num);
}
List<Integer> res = new ArrayList<>();
for (int f = bucket.length - 1; f >= 0 && res.size() < k; f--) {
    if (bucket[f] != null) res.addAll(bucket[f]);
}
return res.subList(0, k);

```

```

(set.contains(cur + 1)) {
    cur++; len++;
}
longest = Math.max(longest, len);
}
return longest;

```

OPERATION	TIME COMPLEXITY
Sort (<code>Arrays.sort</code>)	$O(n \log n)$
Building Map/Set from n items	$O(n)$
Heap push / pop	$O(\log n)$
Bucket Sort (by frequency)	$O(n)$
Sudoku (fixed 9x9)	$O(1)$

7. VALID SUDOKU (3 CHECKS)

Rows	Columns	3x3 Boxes
<pre> for (int i = 0; i < 9; i++) { Set<Character> seen = new HashSet<>(); for (int j = 0; j < 9; j++) { char c = board[i][j]; if (c != '.' && !seen.add(c)) return false; } } </pre>	<pre> for (int j = 0; j < 9; j++) { Set<Character> seen = new HashSet<>(); for (int i = 0; i < 9; i++) { char c = board[i][j]; if (c != '.' && !seen.add(c)) return false; } } </pre>	<pre> for (int br = 0; br < 9; br += 3) { for (int bc = 0; bc < 9; bc += 3) { Set<Character> seen = new HashSet<>(); for (int i = br; i < br + 3; i++) { for (int j = bc; j < bc + 3; j++) { char c = board[i][j]; if (c != '.' && !seen.add(c)) return false; } } } } </pre>

```
} }  
}  
}
```

E. HASHMAP / HASHSET REMINDERS	
HashMap Notes	HashSet Notes
- map.put(key, value) - map.get(key) - map.getOrDefault(key, defaultValue) - map.containsKey(key) - map.containsValue(value) - map.remove(key) - map.size(), map.isEmpty() - map.keySet(), map.values(), map.entrySet()	- set.add(x) → returns false if already exists - set.contains(x) - set.remove(x) - set.size(), set.isEmpty() - set.clear() - set → new ArrayList<set> to convert

F. FREQUENTLY USED STRUCTURES	
Structure	When to Use
HashSet	Check existence, remove duplicates
HashMap	Count frequency, store mapping
int[26]	Lowercase character frequency
PriorityQueue	Top K, Min/Max problems
Queue / Deque	BFS / Sliding Window / Monotonic Queue
Stack	Monotonic Stack, Next Greater, Valid Parentheses

G. TIME COMPLEXITY OF COMMON ALGORITHMS		
Algorithm / Approach	Time Complexity	Best Case / Notes
Brute Force (single loop)	O(n)	—
Brute Force (nested loops)	O(n^2)	—
Hashing (build map/set)	O(1)	O(1) avg per op
Sorting	O(n log n)	Timsort in Java
Two Pointers (sorted)	O(n)	After sorting if needed
Prefix / Suffix	O(n)	Two linear passes
Sliding Window	O(n)	Each element visited at most twice

H. MEMORY USAGE GUIDE	
Structure	Space Usage
HashSet (n elements)	O(n)
HashMap (n pairs)	O(n)
int[26]	O(1)
Array / List (n)	O(n)
Stack / Queue (n)	O(n)
PriorityQueue (n)	O(n)

GOLDEN TIPS

Understand the pattern first, then code

Choose the right data structure

Analyze time & space

Handle edge cases

Write clean & simple code

Practice consistently

ARRAYS & HASHING - JAVA CHEAT SHEET (IMAGE 2)

Arrays + Hashing

1 DATA STRUCTURES - QUICK REFERENCE

HashSet<E> (Unique Elements)

```
Set<Character> set = new
HashSet<>();
set.add('a');        //
true if added
set.contains('a');    //
O(1) avg
set.remove('a');
set.size();
set.isEmpty();
set.clear();
```

Time Complexity:
add/contains/remove → O(1) avg

HashMap<K, V> (Key → Value)

```
Map<String, Integer> map
= new HashMap<>();
map.put("a", 1);
// insert/update
map.get("a");
// returns value or null
map.getDefault("a",
0); // default if not
present
map.containsKey("a");
map.containsValue(1);
map.remove("a");
map.size();
map.isEmpty();
map.clear();
```

Time Complexity:
put/get/containsKey → O(1) avg

LinkedHashMap<K, V> (Maintains Insertion Order)

```
Map<Integer, Integer>
lhm = new
```

2 COMMON JAVA CONVERSIONS

String ↔ char[]

```
String s =
"abc";
char[] arr =
s.toCharArray();
String s2 =
new
String(arr);
```

List ↔ Array

```
List<Integer>
list =
Arrays.asList(
1,2,3);
// Fixed-size
list backed by
array
Integer[] arr
=
list.toArray(new
Integer[0]);
int[] arr2 =
list.stream().
mapToInt(i →
i)
.toArray();
```

Set ↔ List / Array

```
Set<Integer>
set = new
HashSet<>();
set.add(1);
set.add(2);
List<Integer>
list = new
ArrayList<>
(set);
Integer[] arr
=
set.toArray(new
Integer[0]);
```

String ↔ int

```
String s =
"123";
int x =
Integer.parseI
nt(s);
String s2 =
String.valueOf
(x);
// or
String s3 = x
+ "";
```

Array ↔ List (Mutable)

```
Integer[] arr
= {1,2,3};
List<Integer>
list = new
ArrayList<>
(
Arrays.asList(
arr));
```

Map Values / Keys

```
Collection<Int
eger> values =
map.values();
Set<String>
keys =
map.keySet();
List<Integer>
valuesList =
new
ArrayList<>
(
map.values());
```

Math Utilities

```
Math.max(a,
b);
Math.min(a,
b);
Math.abs(x);
Math.pow(a,
```

3 USEFUL JAVA PATTERNS & METHODS

computeIfAbsent() – Very Important!

```
Map<Character, List<Integer>> map
= new HashMap<>();
map.computeIfAbsent('a', k → new
ArrayList<>()).add(1);
// If key 'a' not present,
creates new ArrayList<>
// then adds 1. Else, directly
adds to existing list.
```

getOrDefault()

```
int count = map.getOrDefault(key,
0);
// returns value if key exists,
else default (0)
```

merge() – Another Powerful One

```
map.merge(key, 1, Integer::sum);
// If key absent → put(key, 1)
// If present → oldVal + 1
```

forEach() with Lambda

```
map.forEach((key, val) → {
    System.out.println(key + " →
" + val);
});
```

Stream APIs (Often Used)

```
List<Integer> list = new
ArrayList<>(map.values());
list.stream().sorted().toList();
list.stream().sorted(Comparator.r
everseOrder()).toList();
list.stream().limit(k).toList();
list.stream().mapToInt(i →
i).toArray();
Arrays.stream(arr).boxed().collec
t(Collectors.toList());
```

Arrays Utility

```
Arrays.sort(arr);
Arrays.sort(arr,
Comparator.reverseOrder());
Arrays.fill(arr, 0);
```

```
LinkedHashMap<>();
lhm.put(1, 10);
lhm.put(2, 20);
// Iterates in insertion
order
```

🌲 TreeMap<K, V> (Sorted by Key)

```
Map<Integer, String> tm
= new TreeMap<>();
tm.put(3, "c");
tm.put(1, "a");
tm.put(2, "b");
// Keys iterated in
sorted order: 1, 2, 3
```

Time Complexity: $O(\log n)$

🔑 PriorityQueue<E> (Min-Heap by default)

```
PriorityQueue<Integer>
pq = new PriorityQueue<>();
pq.add(5); pq.add(1);
pq.add(3);
pq.peek();    // 1 (min)
pq.poll();    // 1
pq.size();
pq.isEmpty();

// For Max-Heap:
PriorityQueue<Integer>
pqMax = new
PriorityQueue<>(
Collections.reverseOrder());
```

Time Complexity: add/poll/peek → $O(\log n)$

StringBuilder

(for Encoding / Building Strings)

```
StringBuilder
sb = new
StringBuilder(
);
sb.append('a')
;
sb.append("bc"
);
sb.append(123)
;
sb.length();
sb.charAt(0);
sb.toString();
sb.setLength(0)
; // clear
```

```
b);
Math.floor(x);
Math.ceil(x);
```

```
Arrays.copyOf(arr, newLength);
Arrays.equals(arr1, arr2);
```

Queue / Deque

```
Queue<Integer> q = new LinkedList<>();
q.offer(1); q.poll(); q.peek();
Deque<Integer> dq = new
ArrayDeque<>();
dq.offerFirst(1); dq.offerLast(2);
dq.pollFirst(); dq.pollLast();
dq.peekFirst(); dq.peekLast();
```

Stack (Legacy) or Deque (Preferred)

```
Stack<Integer> st = new Stack<>();
st.push(1); st.pop(); st.peek();
st.isEmpty();
Deque<Integer> st2 = new
ArrayDeque<>(); // better
st2.push(1); st2.pop(); st2.peek();
```

6 TOP K FREQUENT (HashMap - + PQ) TEMPLATE

```
Map<Integer, Integer>
freq = new HashMap<>();
for (int n : nums) {
    freq.merge(n, 1,
Integer::sum);
}
PriorityQueue<int[]>
pq = new
PriorityQueue<>(
(a, b) → a[1] -
b[1] // min-heap by
freq
);
for (int[] e :
freq.entrySet().stream()
.map(e → new
int[]{e.getKey(),
e.getValue()}))
.toList()) {
    pq.offer(e);
    if (pq.size() >
k) pq.poll();
}
// Extract from pq
(smallest to largest
freq)
int[][] ans = new
int[k][2];
for (int i = k - 1; i
≥ 0; i--) ans[i] =
pq.poll();
```

7 CHECKLIST - WHEN SOLVING

- ✓ Understand constraints (n, values, range).
- ✓ Think: Can I use HashSet for $O(1)$ lookups?
- ✓ Need counts? → Use HashMap / int[26]
- ✓ Need order? → LinkedHashMap / TreeMap
- ✓ Top K / Min/Max? → PriorityQueue (Heap)
- ✓ Need grouping? → HashMap<Key, List>
- ✓ Need encoding? → length#string
- ✓ Can I do prefix/suffix to avoid re-computation?
- ✓ Always analyze time & space complexity.

4 FREQUENCY (For Lower Letters)

```
int[] freq =
new int[26];

for (char c :
s.toCharArray()) {
    freq[c -
'a']++;
}

// index = char
- 'a' (0 to
25)
// Works for
'a' to 'z' only
```

5 BUCKET / GROUPING PATTERN

```
Map<String,
List<String>>
map = new
HashMap<>();

for (String s :
strs) {
    char[] arr
=
s.toCharArray();
Arrays.sort(arr);
String key
= new
String(arr);

map.computeIfAbsent(key, k →
new ArrayList<>()).add(s);
}

List<List<String>> res = new
ArrayList<>(map.values());
```

8 QUICK SYNTAX REMINDER(One-Liners You Use Often)

Iterate HashMap

```
for (Map.Entry<K, V> e :
map.entrySet()) {
    K key = e.getKey();
```

Iterate HashSet

```
for (E e : set) {
    // use e
}
```

Create List

```
List<Integer> list = new
ArrayList<>();
List<Integer> list2 =
Arrays.asList(1,2,3);
```

Convert List to Array

```
Integer[] arr =
list.toArray(new
Integer[0]);
int[] arr2 =
list.stream().mapToInt(i →
```

```
V val = e.getValue();  
}
```

```
List<Integer> list3 = new  
ArrayList<>(list2);
```

```
i)  
    .toArray();
```

Sort List

```
Collections.sort(list);  
list.sort(Comparator.reverseOrder())  
;
```

Reverse String

```
new StringBuilder(s)  
    .reverse().toString();
```

• MOST USED IN INTERVIEWS

- HashSet
- HashMap + computeIfAbsent
- getOrDefault / merge
- PriorityQueue
- Arrays.sort(), toArray(), asList()
- StringBuilder

★
REMEMBER

✿ Pattern Recognition >
Code

🏆 Know your
Tools

💡 Write Clean & Simple
Code

🔍 Analyze
Complexity

🕒 Practice
Consistently